

Erlind Sejdiu
106357

Exposé: Verbesserung der Analysierbarkeit der HKA-App durch OpenTelemetry – eine Fallstudie nach ISO/IEC 25010

Projektarbeit
der Hochschule Karlsruhe
im Master Studiengang Informatik
im Wintersemester 2026/27

Korrektor: [Name der betreuenden Person]

Inhaltsverzeichnis

1	Motivation	2
2	Problemstellung	2
3	Stand der Technik	3
4	Forschungsfrage	4
5	Methodik	4
5.1	Einführung von OpenTelemetry	4
5.2	Kontrolliertes Experiment zur Fehlerdiagnose (TF1)	5
5.3	Befragung (TF2)	5
5.4	Monitoring (TF4)	5
5.5	Datenschutz	5
5.6	Validität	6
6	Zeitplan	6
	Literaturverzeichnis	6

1 Motivation

Die HKA-App ist die zentrale mobile Anwendung der Hochschule Karlsruhe und wird im Rahmen studentischer Projektarbeiten kontinuierlich weiterentwickelt. Sie besteht aus mehreren Client-Anwendungen für Android, iOS, Windows und das Web sowie aus einem gemeinsam genutzten Backend, das von eigenen Server- und Admin-Teams betreut wird. Eine einzelne Nutzeraktion, etwa das Abrufen des Mensaplans oder des Stundenplans, durchläuft damit mehrere Komponenten: den jeweiligen Client, das Netzwerk, den Server sowie gegebenenfalls externe Datenquellen der Hochschule.

Tritt in einem solchen System ein Fehler auf oder reagiert die App langsam, ist die Ursache häufig schwer zu lokalisieren. Klassisches Logging erfasst jeweils nur die Sicht einer einzelnen Komponente, und die Log-Einträge von Client und Server lassen sich nicht ohne Weiteres einer gemeinsamen Anfrage zuordnen. Verstärkt wird dies dadurch, dass die Teams der HKA-App semesterweise wechseln: Wissen über das Laufzeitverhalten des Systems geht dabei leicht verloren, und neue Studierende müssen sich bei jedem Fehler erneut in fremden Code und fremde Logs einarbeiten.

Unter dem Begriff *Observability* werden Verfahren zusammengefasst, die den inneren Zustand eines Systems anhand seiner Ausgaben – Traces, Metriken und Logs – nachvollziehbar machen [1]. OpenTelemetry hat sich als herstellerunabhängiger De-facto-Standard hierfür etabliert und unterstützt neben gängigen Backend-Sprachen auch Browser-, Android- und iOS-Anwendungen [2], [3]. Damit ist es grundsätzlich möglich, eine Anfrage vom Client bis in das Backend als zusammenhängenden Trace sichtbar zu machen. Ob dies die Qualität einer Software im Sinne eines anerkannten Qualitätsmodells tatsächlich verbessert, ist die Ausgangsfrage dieser Arbeit.

2 Problemstellung

Das Qualitätsmodell ISO/IEC 25010 beschreibt die Produktqualität von Software anhand von Qualitätsmerkmalen und zugehörigen Teilmerkmalen [4]. OpenTelemetry verbessert dabei nicht pauschal „die Qualität“ einer Anwendung. Der unmittelbare Bezug besteht zum Teilmerkmal *Analysierbarkeit* (engl. *Analysability*) des Merkmals Wartbarkeit: Es beschreibt, wie effektiv und effizient sich die Auswirkungen von Änderungen abschätzen sowie Mängel und Fehlerursachen diagnostizieren lassen. Indirekt kann eine schnellere Fehlerdiagnose zudem die *Zuverlässigkeit* verbessern, da Störungen schneller erkannt und behoben werden. Gleichzeitig wirkt sich die Instrumentierung negativ auf die *Leistungseffizienz* aus, da sie zusätzliche Rechenzeit, Speicher und Netzwerkbandbreite benötigt. Die Einführung von OpenTelemetry stellt damit einen Zielkonflikt zwischen Qualitätsmerkmalen dar.

Für die HKA-App ergeben sich daraus folgende Probleme:

1. Es ist nicht bekannt, ob und in welchem Maße OpenTelemetry die Analysierbarkeit der HKA-App verbessert, und der Nutzen lässt sich ohne konkrete Messgrößen nicht belegen.
2. Die Einführung selbst erfordert Implementierungsaufwand in mehreren Codebasen und eine Reihe von Architekturentscheidungen (Instrumentierungsart, Analyse-Backend, Sampling), für die Alternativen abgewogen werden müssen.

3. Da die App von Studierenden genutzt wird, müssen Telemetriedaten datenschutzkonform (DSGVO) erhoben werden.

Abgrenzung. Die Quantifizierung des Laufzeit- und Datenoverheads von OpenTelemetry ist Gegenstand einer separaten Seminararbeit des Autors [Titel der Seminararbeit]. Deren Ergebnisse fließen in die Konfigurationsentscheidungen dieser Arbeit ein und werden zitiert, der Overhead wird hier jedoch nicht erneut systematisch untersucht, sondern lediglich stichprobenartig im realen System überprüft. Ebenfalls nicht Ziel der Arbeit ist die vollständige Instrumentierung aller vier Clients: Instrumentiert werden das Backend, die Web-App und der Android-Client; die Übertragbarkeit auf iOS und Windows wird diskutiert.

3 Stand der Technik

Verteiltes Tracing. Die Grundlagen des verteilten Tracings wurden maßgeblich durch Arbeiten wie X-Trace [5] und Googles Dapper [6] gelegt, deren Modell aus Traces und Spans die Grundlage von OpenTelemetry bildet. Die Weitergabe des Trace-Kontexts zwischen Komponenten ist durch den W3C-Standard *Trace Context* vereinheitlicht [7]. Mit Canopy beschreibt Facebook ein System, das Traces vom Browser bzw. der mobilen App bis in das Backend verfolgt, und zeigt, dass gerade die Verbindung von Client- und Serversicht für die Analyse von Problemen wertvoll ist [8]. Diese Arbeiten beziehen sich jedoch auf Infrastrukturen großer Unternehmen, deren Größenordnung und Organisationsform mit einer Hochschul-App nicht vergleichbar sind.

OpenTelemetry auf Clients. OpenTelemetry stellt SDKs für Browser, Android und iOS bereit [9], [10], [11]. Die Dokumentation beschreibt überwiegend qualitativ, *wie* instrumentiert werden kann; die mobilen SDKs befinden sich teilweise noch in Entwicklung, sodass Erfahrungswerte rar sind.

Qualitätsmodelle und Messgrößen. ISO/IEC 25010 definiert die Qualitätsmerkmale selbst, jedoch keine Messverfahren [4]. Messgrößen für die einzelnen Teilmerkmale werden in ISO/IEC 25023 vorgeschlagen, für die Analysierbarkeit unter anderem Maße zur Vollständigkeit der Systemprotokollierung und zur Wirksamkeit von Diagnosefunktionen [12]. [Genaue Bezeichnungen der Analysierbarkeits-Maße in ISO/IEC 25023 prüfen.] Aus der DevOps-Praxis ist zudem die *Time to Restore Service* bzw. Mean Time to Recovery als Kennzahl für die Fähigkeit bekannt, Störungen zu beheben [13]. Für das Monitoring im Betrieb haben sich Service Level Indicators und Objectives (SLIs/SLOs) etabliert [14].

Forschungslücke. Die Konzepte des verteilten Tracings sind gut untersucht, und die Vorteile von Observability werden in der Praxisliteratur häufig betont [1]. Empirische Untersuchungen, die den Effekt auf die Fehlerdiagnose anhand definierter Messgrößen belegen, sind hingegen selten – insbesondere für kleine, heterogene Systeme mit mehreren nativen Clients und wechselnden Entwicklerteams. [Literatur zu empirischen Studien über Debugging mit Traces vs. Logs recherchieren und ergänzen.]

4 Forschungsfrage

Aus der Problemstellung und dem Stand der Technik leitet sich folgende zentrale Forschungsfrage ab:

Wie verändert die Einführung von OpenTelemetry die Analysierbarkeit der HKA-App im Sinne von ISO/IEC 25010, gemessen an Erfolg und Dauer der Fehlerdiagnose, und welcher Aufwand steht dem gegenüber?

Zur Beantwortung wird die Forschungsfrage in folgende Teilfragen gegliedert:

- TF1:** Wie verändern sich Diagnoseerfolg und Diagnosedauer bei Fehlern in der HKA-App, wenn Entwickler neben Logs auch Traces zur Verfügung haben?
- TF2:** Wie bewerten die Entwicklerinnen und Entwickler der HKA-App die Analysierbarkeit des Systems vor und nach der Einführung?
- TF3:** Welcher Implementierungs- und Konfigurationsaufwand fällt je Plattform an, und welche Architekturentscheidungen sind dabei zu treffen?
- TF4:** (*Nebenfrage*) Inwieweit lassen sich auf Basis der erhobenen Telemetrie sinnvolle SLIs und ein Monitoring der HKA-App aufbauen?

5 Methodik

Die Arbeit ist als Fallstudie an der realen HKA-App angelegt, die durch ein kontrolliertes Experiment (TF1) und eine Befragung (TF2) ergänzt wird.

5.1 Einführung von OpenTelemetry

Instrumentiert werden das Backend [Technologie des HKA-Backends eintragen], die Web-App und der Android-Client. Die Telemetriedaten werden über OTLP an einen OpenTelemetry Collector gesendet und in einem Analyse-Backend für Traces, Metriken und Logs ausgewertet. Die Einführung erfolgt schrittweise: zunächst automatische Instrumentierung des Backends, anschließend durchgängige Traces vom Client bis in das Backend über W3C Trace Context, schließlich gezielte manuelle Spans und die Korrelation von Logs mit Trace-IDs. Granularität und Sampling-Rate werden auf Basis der Ergebnisse der Seminararbeit gewählt.

Architekturentscheidungen (TF3). Alle wesentlichen Entscheidungen werden als *Architecture Decision Records* (ADR) dokumentiert [15], jeweils mit betrachteten Alternativen, Bewertungskriterien und Begründung. Mindestens folgende Entscheidungen werden getroffen:

- OpenTelemetry vs. bestehende Logging-Lösung bzw. spezialisierte Dienste (z.B. Sentry, Firebase Crashlytics),
- automatische vs. manuelle Instrumentierung und Wahl der Granularität,
- Analyse-Backend (z.B. Jaeger vs. Grafana Tempo mit Prometheus und Loki),
- Sampling-Strategie (Head- vs. Tail-Sampling) und Sampling-Rate,
- Umgang mit personenbezogenen Daten.

Der Implementierungsaufwand wird je Plattform über hinzugefügte Codezeilen, Konfigurationsdateien, Abhängigkeiten und aufgewendete Arbeitszeit erfasst.

5.2 Kontrolliertes Experiment zur Fehlerdiagnose (TF1)

Die Analysierbarkeit wird anhand von Fehlerszenarien gemessen. Dazu werden in einer Testumgebung gezielt Fehler in die HKA-App eingebracht (Fault Injection), z. B. eine verzögerte Datenbankabfrage, ein fehlschlagender Aufruf einer externen Datenquelle, ein fehlerhaftes Datenformat zwischen Client und Server oder ein Fehler, der nur auf einem Client auftritt. Teilnehmende aus den Teams der HKA-App (angestrebt: 6–10 Personen) erhalten jeweils eine Fehlerbeschreibung aus Nutzersicht und sollen die Ursache finden.

Es wird ein *Within-Subject*-Design gewählt: Jede Person bearbeitet die Hälfte der Szenarien nur mit Logs (Kontrollbedingung) und die andere Hälfte mit Logs und Traces (Experimentalbedingung). Die Zuordnung von Szenarien und Reihenfolge wird über ein lateinisches Quadrat ausbalanciert, um Lern- und Reihenfolgeeffekte zu verringern. Jedes Szenario ist auf eine feste Zeit begrenzt. Erhoben werden:

- **Diagnoseerfolg:** Anteil der Szenarien, in denen die Ursache korrekt identifiziert wurde,
- **Diagnosedauer:** Zeit bis zur korrekten Identifikation der Ursache,
- **Diagnoseaufwand:** Anzahl der verwendeten Werkzeuge bzw. der betrachteten Komponenten.

Diese Größen werden den Analysierbarkeits-Maßen nach ISO/IEC 25023 zugeordnet [12]. Zusätzlich wird analog die Abdeckung der Instrumentierung erfasst, d. h. der Anteil der Anwendungsfälle, die als vollständiger Trace sichtbar sind.

5.3 Befragung (TF2)

Vor der Einführung und nach dem Experiment werden die Mitglieder der HKA-App-Teams mit einem kurzen Fragebogen (Likert-Skala) zur wahrgenommenen Analysierbarkeit und Nützlichkeit befragt, ergänzt um offene Fragen zu Hindernissen und Verbesserungsvorschlägen.

5.4 Monitoring (TF4)

Sofern ein Einsatz im Produktivbetrieb mit den Verantwortlichen der HKA-App vereinbart werden kann, werden für zentrale Anwendungsfälle SLIs (z. B. Verfügbarkeit und 95. Perzentil der Antwortzeit) und SLOs definiert [14] und in Dashboards und Alarmen umgesetzt. Ist kein Produktivbetrieb möglich, erfolgt dies in der Testumgebung unter synthetischer Last, und die Aussagekraft wird entsprechend eingeschränkt diskutiert. [Mit den Verantwortlichen der HKA-App klären, ob ein Produktivbetrieb möglich ist.]

5.5 Datenschutz

Es werden keine personenbezogenen Daten wie Matrikelnummern, Namen oder vollständige IP-Adressen als Attribute erfasst. Das Experiment findet in einer Testumgebung mit Testkonten statt. Die Teilnahme an Experiment und Befragung ist freiwillig, die Auswertung erfolgt anonymisiert.

5.6 Validität

Die Aussagekraft ist durch die geringe Zahl an Teilnehmenden, die künstlich eingebrachten Fehler und die Rolle des Autors als Entwickler der Instrumentierung und Versuchsleiter begrenzt. Die Ergebnisse sind daher als explorativ zu verstehen. Diese Einschränkungen werden in der Ausarbeitung systematisch diskutiert.

6 Zeitplan

Den einzelnen Phasen der Arbeit sind konkrete Ziele und Artefakte zugeordnet:

Phase 1: Einarbeitung und Ist-Analyse

Ziel: Verständnis der Architektur der HKA-App und des vorhandenen Loggings, Abstimmung mit den Teams, Vorab-Befragung.

Artefakt: Architekturübersicht, Liste der betrachteten Anwendungsfälle, Ergebnisse der Vorab-Befragung.

Phase 2: Architekturentscheidungen und Infrastruktur

Ziel: Abwägung der Alternativen und Aufbau von Collector und Analyse-Backend in einer Testumgebung.

Artefakt: ADRs, lauffähige Observability-Infrastruktur.

Phase 3: Instrumentierung von Backend und Clients

Ziel: Instrumentierung von Backend, Web-App und Android-Client mit durchgängigen Traces und Log-Korrelation.

Artefakt: Instrumentierte Codebasen, Dokumentation des Implementierungsaufwands.

Phase 4: Experiment und Befragung

Ziel: Vorbereitung der Fehlerszenarien, Durchführung des Experiments und der Abschlussbefragung.

Artefakt: Fehlerszenarien, Rohdaten des Experiments und der Befragung.

Phase 5: Monitoring

Ziel: Definition von SLIs/SLOs, Dashboards und Alarmen.

Artefakt: Monitoring-Konfiguration.

Phase 6: Evaluation und Ausarbeitung

Ziel: Auswertung, Beantwortung der Forschungsfrage, Handlungsempfehlung für die HKA-App.

Artefakt: Schriftliche Ausarbeitung, dokumentierter Quellcode im Git-Repository, Leitfaden für nachfolgende Teams.

Literaturverzeichnis

- [1] C. Majors, L. Fong-Jones und G. Miranda, *Observability Engineering: Achieving Production Excellence*. Sebastopol, CA: O'Reilly Media, 2022.

- [2] Cloud Native Computing Foundation. „OpenTelemetry – Project Page,“ besucht am 28. Sep. 2026. Adresse: <https://www.cncf.io/projects/opentelemetry/>
- [3] OpenTelemetry Authors. „OpenTelemetry Documentation,“ besucht am 28. Sep. 2026. Adresse: <https://opentelemetry.io/docs/>
- [4] *ISO/IEC 25010:2023 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*, International Organization for Standardization, 2023.
- [5] R. Fonseca, G. Porter, R. H. Katz, S. Shenker und I. Stoica, „X-Trace: A Pervasive Network Tracing Framework,“ in *4th USENIX Symposium on Networked Systems Design & Implementation (NSDI 07)*, 2007.
- [6] B. H. Sigelman u. a., „Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,“ Google, Inc., Techn. Ber. dapper-2010-1, 2010.
- [7] World Wide Web Consortium. „Trace Context – W3C Recommendation,“ besucht am 28. Sep. 2026. Adresse: <https://www.w3.org/TR/trace-context/>
- [8] J. Kaldor u. a., „Canopy: An End-to-End Performance Tracing And Analysis System,“ in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*, 2017, S. 34–50. DOI: 10.1145/3132747.3132749
- [9] OpenTelemetry Authors. „OpenTelemetry JavaScript – Browser,“ besucht am 28. Sep. 2026. Adresse: <https://opentelemetry.io/docs/languages/js/getting-started/browser/>
- [10] OpenTelemetry Authors. „OpenTelemetry Android,“ besucht am 28. Sep. 2026. Adresse: <https://github.com/open-telemetry/opentelemetry-android>
- [11] OpenTelemetry Authors. „OpenTelemetry Swift,“ besucht am 28. Sep. 2026. Adresse: <https://github.com/open-telemetry/opentelemetry-swift>
- [12] *ISO/IEC 25023:2016 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Measurement of system and software product quality*, International Organization for Standardization, 2016.
- [13] N. Forsgren, J. Humble und G. Kim, *Accelerate: The Science of Lean Software and DevOps*. Portland, OR: IT Revolution Press, 2018.
- [14] B. Beyer, C. Jones, J. Petoff und N. R. Murphy, Hrsg., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media, 2016.
- [15] M. Nygard. „Documenting Architecture Decisions,“ besucht am 28. Sep. 2026. Adresse: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>